NeuralNetwork

Generated by Doxygen 1.15.0

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

Chapter 3

Class Documentation

3.1 ActivationFunction Struct Reference

Wrapper for an activation function and its derivative. [.

`#include <math_functions.h>`

Public Attributes

- std::function< std::vector< float >(const std::vector< float > &)> **func**
- std::function< std::vector< float >(const std::vector< float > &)> **derivative**

3.1.1 Detailed Description

Wrapper for an activation function and its derivative. [.

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/math_functions.h

3.2 ColumnData Struct Reference

Metadata for dataset column.

`#include <data_preprocessor.h>`

Public Attributes

- bool **isNumeric**
- std::vector< std::string > **categories**
- std::unordered_map< std::string, int > **category_to_index**
- float **range** [2]

3.2.1 Detailed Description

Metadata for dataset column.

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/data_preprocessor.h

3.3 Configuration Struct Reference

Holds all runtime settings for the neural network tool.

Public Attributes

- std::string **initialization_method** = "Xavier"
- int **hidden_layers** = 1
- int **neurons_nb** = 2
- std::vector< std::string > **activations** = {}
- std::string **train_data_path** = "iris.csv"
- std::string **target_column** = "species"
- float **test_fraction** = 0.2f
- int **epochs** = 140
- float **learning_rate** = 0.3f
- std::string **model_save_path** = ""
- std::string **model_load_path** = ""

3.3.1 Detailed Description

Holds all runtime settings for the neural network tool.

This structure is populated by the command-line parser and determines the architecture, data paths, and training hyperparameters.

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/main.cpp

3.4 DataPreprocessor Class Reference

Handles the conversion of CSV string data into normalized matrices of floats.

```
#include <data_preprocessor.h>
```

Public Member Functions

- void **clear** ()

  Clears all the member variables of the DataPreprocessor.
- void fit (const std::vector< std::vector< std::string > > &data)

  Fits the preprocessor by determining column types and calculating normalization ranges.
- std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > > transform_and_extract (const std::vector< std::vector< std::string > > &data, const std::string &column_to_extract, bool is_there↩ _header=true)

  Transforms the dataset using One-Hot encoding and Min-Max normalization.
- void interpret_extracted_column (const std::vector< float > &transformed_data) const

  Decodes processed float data back into human-readable format.
- void **print_state** () const

  Prints the current state of the DataPreprocessor.

3.4.1 Detailed Description

Handles the conversion of CSV string data into normalized matrices of floats.

3.4.2 Member Function Documentation

3.4.2.1 fit()

```
void DataPreprocessor::fit (
            const std::vector< std::vector< std::string > > & data)
```

Fits the preprocessor by determining column types and calculating normalization ranges.

Iterates through the dataset to identify which column is numeric and which is categorical. For numeric columns, it finds min/max for normalization. For categorical, it maps unique strings to indices.

Parameters

data	Dataset (2D string vector). The first row must be the header.

Note

This function populates internal maps and clears any previous state.

3.4.2.2 interpret_extracted_column()

```
void DataPreprocessor::interpret_extracted_column (
            const std::vector< float > & transformed_data) const
```

Decodes processed float data back into human-readable format.

Reverses normalization for numeric columns and picks the highest probability category for one-hot encoded categorical columns.

Parameters

transformed_data	A 1D vector representing a single sample of the extracted column.

Precondition

transform_and_extract() must have been called at least once to set the extracted column name.

3.4.2.3 transform_and_extract()

```
std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > >
DataPreprocessor::transform_and_extract (
            const std::vector< std::vector< std::string > > & data,
            const std::string & column_to_extract,
            bool is_there_header = true)
```

Transforms the dataset using One-Hot encoding and Min-Max normalization.

Separates the dataset into a feature set and a specific target column.

Parameters

data	The raw string data to transform.
column_to_extract	The name of the column to be used as the target.
is_there_header	Flag indicating if the input dataset includes a header row.

Returns

A pair of 2D float vectors:

- **first**: Transformed feature matrix (all columns except the extracted one).
- **second**: Transformed target matrix (the extracted column).

Precondition

The preprocessor must have been successfully fitted using fit().

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/data_preprocessor.h
- NeuralNetworkProjectFoCP/data_preprocessor.cpp

3.5 LossFunction Struct Reference

Wrapper for a loss function and its derivative.

`#include <math_functions.h>`

Public Attributes

- std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)> **func**
- std::function< std::vector< float >(const std::vector< float > &, const std::vector< float > &)> **derivative**

3.5.1 Detailed Description

Wrapper for a loss function and its derivative.

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/math_functions.h

3.6 NeuralNetwork Class Reference

Implementation of a neural network with multiple functionalities.

`#include <neural_network.h>`

Public Member Functions

- void initialize_weights_and_biases (int input_size, int hidden_layers_number, int neurons_per_hidden_layer, int output_size, std::string initialization_method="Xavier", std::vector< std::string > activation_functions={}, int seed=std::random_device()())

  Initialize weights and biases for the neural network.
- void **visualize_model** () const

  Visualize the neural network model structure (weights and biases).
- std::vector< std::vector< float > > feedforward (const std::vector< std::vector< float > > &input) const

  Performs a forward pass through the network for multiple input samples.
- void train (const std::vector< std::vector< float > > &inputs, const std::vector< std::vector< float > > &targets, const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, int epochs, float learning_rate, const std::string &loss_function_name)

  Train the neural network using the provided training data.
- float test_model (const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, const std::string &loss_function_name) const

  Calculates the network loss on a test dataset.
- void save_model_to_file (const std::string &filename) const

  Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.
- void load_model_from_file (const std::string &filename)

  Loads the model state (weights, biases, activations) from files.
- const std::vector< std::vector< std::vector< float > > > & **get_weights** () const
- const std::vector< std::vector< float > > & **get_biases** () const
- const std::vector< std::string > & **get_activations** () const

3.6.1 Detailed Description

Implementation of a neural network with multiple functionalities.

3.6.2 Member Function Documentation

3.6.2.1 feedforward()

```
std::vector< std::vector< float > > NeuralNetwork::feedforward (
            const std::vector< std::vector< float > > & input) const
```

Performs a forward pass through the network for multiple input samples.

Parameters

input	A 2D vector where each row is an input sample.

Returns

A 2D vector of predictions. Returns an empty 2D vector if the model is invalid.

Note

Ensure the input sample size matches the network's input layer size.

3.6.2.2 initialize_weights_and_biases()

```
void NeuralNetwork::initialize_weights_and_biases (
            int input_size,
            int hidden_layers_number,
            int neurons_per_hidden_layer,
            int output_size,
            std::string initialization_method = "Xavier",
            std::vector< std::string > activation_functions = {},
            int seed = std::random_device()())
```

Initialize weights and biases for the neural network.

Supports Xavier and He initialization methods. Biases are initialized to zero.

Parameters

input_size	The size of the input layer.
hidden_layers_number	The number of hidden layers.
neurons_per_hidden_layer	The number of neurons in each hidden layer.
output_size	The size of the output layer.

initialization_method	The method used ("Xavier" or "He"). Defaults is Xavier.
activation_functions_passed	Activation functions for each layer (count should be hidden_layers_number + 1).
seed	The random seed for reproducibility.

3.6.2.3 load_model_from_file()

```
void NeuralNetwork::load_model_from_file (
            const std::string & filename)
```

Loads the model state (weights, biases, activations) from files.

Parameters

filename	The base name of the files (appends _weights.txt etc automatically).

Note

This function calls validate_model_structure() after loading.

3.6.2.4 save_model_to_file()

```
void NeuralNetwork::save_model_to_file (
            const std::string & filename) const
```

Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.

Parameters

filename	The base name of the files (appends _weights.txt etc. automatically).

3.6.2.5 test_model()

```
float NeuralNetwork::test_model (
            const std::vector< std::vector< float > > & test_inputs,
            const std::vector< std::vector< float > > & test_targets,
            const std::string & loss_function_name) const
```

Calculates the network loss on a test dataset.

Parameters

test_inputs	Input samples for testing.
test_targets	Expected output labels.

loss_function_name	The name of the loss function ("MSE" or "BCE").

Returns

The average loss across all samples. Returns -1.0f if the model is invalid or sizes mismatch.

3.6.2.6 train()

```
void NeuralNetwork::train (
            const std::vector< std::vector< float > > & inputs,
            const std::vector< std::vector< float > > & targets,
            const std::vector< std::vector< float > > & test_inputs,
            const std::vector< std::vector< float > > & test_targets,
            int epochs,
            float learning_rate,
            const std::string & loss_function_name)
```

Train the neural network using the provided training data.

Parameters

inputs	The input data for training - each row represents a training sample. It is expected to be shuffled.
targets	The target output data for training - each row represents the target output for the corresponding training sample.
test_inputs	The input data for testing during training - each row represents a different test sample.
test_targets	The target output data for testing during training - each row represents the target output for the corresponding test sample.
epochs	The number of epochs to train for.
learning_rate	The learning rate for weight updates.
loss_function_name	The name of the loss function to use - either "MSE" or "BCE".

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/neural_network.h
- NeuralNetworkProjectFoCP/neural_network.cpp

Chapter 4

File Documentation

4.1 NeuralNetworkProjectFoCP/data_preprocessor.h File Reference

Logic for dataset preparation for use in neural network - normalization, and one-hot encoding.

```
#include <vector>
#include <string>
#include <unordered_map>
```

Classes

- struct ColumnData

  Metadata for dataset column.
- class DataPreprocessor

  Handles the conversion of CSV string data into normalized matrices of floats.

4.1.1 Detailed Description

Logic for dataset preparation for use in neural network - normalization, and one-hot encoding.

4.2 data_preprocessor.h

Go to the documentation of this file.

```
00001 #ifndef DATAPREPROCESSOR_H
00002 #define DATAPREPROCESSOR_H
00006 #include <vector>
00007 #include <string>
00008 #include <unordered_map>
00009
00014 struct ColumnData {
00015     bool isNumeric;
00016     std::vector<std::string> categories;      // All unique categories for categorical columns
00017     std::unordered_map<std::string, int> category_to_index;
00018     float range[2];                           // Min and Max for numeric columns
00019 };
00024 class DataPreprocessor {
00025 private:
00026     std::vector<ColumnData> columns;  // Column index -> column_data
```

```
00027     std::unordered_map<std::string, int> column_order;
00028     std::string extracted_column_name;
00029     bool is_fitted = false;
00030
00031 public:
00035     void clear();
00043     void fit(const std::vector<std::vector<std::string»& data);
00055     std::pair<std::vector<std::vector<float», std::vector<std::vector<float»>
      transform_and_extract(const std::vector<std::vector<std::string»& data, const std::string&
      column_to_extract, bool is_there_header =true);
00063     void interpret_extracted_column(const std::vector<float>& transformed_data) const;
00067     void print_state() const;
00068 };
00069
00070 #endif // DATAPREPROCESSOR_H
```

4.3 NeuralNetworkProjectFoCP/display.h File Reference

Header file for display functions.

```
#include <string>
```

Functions

- void print_header (const std::string &text)

  Prints a stylized header to the console.
- void print_vector (const std::vector< float > &vec)

  Prints a 1D vector of floats in a formatted horizontal array.
- void print_feedforward_output (const std::vector< std::vector< float > > &feedforward_input, const std::vector< std::vector< float > > &target_output, const std::vector< std::vector< float > > &feedforward_output, const size_t sample_count=5)

  Displays a comparison between inputs, targets, and actual model predictions.

4.3.1 Detailed Description

Header file for display functions.

4.3.2 Function Documentation

4.3.2.1 print_feedforward_output()

```
void print_feedforward_output (
            const std::vector< std::vector< float > > & feedforward_input,
            const std::vector< std::vector< float > > & target_output,
            const std::vector< std::vector< float > > & feedforward_output,
            const size_t sample_count = 5)
```

Displays a comparison between inputs, targets, and actual model predictions.

Useful for visually inspecting model performance on a small sample of data.

Parameters

feedforward_input	The original feature data.
target_output	The true, target labels.
feedforward_output	The predictions of the NeuralNetwork.
sample_count	The maximum number of samples to print (5 is default).

4.3.2.2 print_header()

```
void print_header (
            const std::string & text)
```

Prints a stylized header to the console.

The header is surrounded by "=" characters from upper and lower side.

Parameters

text	The string to be displayed within the header.

4.3.2.3 print_vector()

```
void print_vector (
            const std::vector< float > & vec)
```

Prints a 1D vector of floats in a formatted horizontal array.

Output format is `[ a b c ... ]` with each value fixed to 2 decimal places and a width of 6 characters for alignment.

Parameters

vec	The vector of floats to display.

4.4 display.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #ifndef DISPLAY_H
00003 #define DISPLAY_H
00004
00005 #include <string>
00009
00015 void print_header(const std::string& text);
00022 void print_vector(const std::vector<float>& vec);
00023
00032 void print_feedforward_output(const std::vector<std::vector<float»& feedforward_input,
00033                               const std::vector<std::vector<float»& target_output,
00034                               const std::vector<std::vector<float»& feedforward_output,
00035                               const size_t sample_count = 5);
00036 #endif
```

4.5 NeuralNetworkProjectFoCP/file_data_operations.h File Reference

Declaration of functions for parsing, loading, and saving data to files.

```
#include <vector>
#include <string>
#include <fstream>
```

Functions

- std::vector< std::string > get_strings_from_line (const std::string &line)

  Convert a line of comma-separated values into a vector of strings.
- std::vector< float > get_floats_from_line (const std::string &line)

  Get a vector of floats from a comma-separated line.
- std::vector< std::vector< std::string > > parseCSV (const std::string &filename)

  Loading CSV file and parsing data into a string matrix.
- void save_vector_to_file (const std::string &filename, const std::vector< float > &vec, char delimiter=',')

  Save a 1D vector of floats to a file.
- void **save_vector_to_file** (const std::string &filename, const std::vector< std::string > &vec, char delimiter=',')

  Save a 1D vector of strings to a file. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.
- void save_vector_to_file (const std::string &filename, const std::vector< std::vector< float > > &vec, char delimiter=',')

  Save a 2D vector of floats to a file.
- void save_vector_to_file (const std::string &filename, const std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')

  Save a 3D vector of floats to a file.
- void load_vector_from_file (const std::string &filename, std::vector< std::string > &vec, char delimiter=',')

  Load a 1D vector of strings from a single line file.
- void load_vector_from_file (const std::string &filename, std::vector< std::vector< float > > &vec, char delimiter=',')

  Load a 2D vector of floats from a file.
- void load_vector_from_file (const std::string &filename, std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')

  Load a 3D vector from a file.

4.5.1 Detailed Description

Declaration of functions for parsing, loading, and saving data to files.

4.5.2 Function Documentation

4.5.2.1 get_floats_from_line()

```
std::vector< float > get_floats_from_line (
            const std::string & line)
```

Get a vector of floats from a comma-separated line.

Parameters

line	String containing comma-separated float values.

Returns

1D vector of floats.

4.5.2.2 get_strings_from_line()

```
std::vector< std::string > get_strings_from_line (
            const std::string & line)
```

Convert a line of comma-separated values into a vector of strings.

Parameters

line	String line containing comma-separated values.

Returns

1D vector of strings.

4.5.2.3 load_vector_from_file() [1/3]

```
void load_vector_from_file (
            const std::string & filename,
            std::vector< std::string > & vec,
            char delimiter = ',')
```

Load a 1D vector of strings from a single line file.

Parameters

	filename	The name of the file to read (".txt" must be included).
out	*vec*	The string vector to be populated (it is cleared before).
	delimiter	The character used to split the values.

4.5.2.4 load_vector_from_file() [2/3]

```
void load_vector_from_file (
            const std::string & filename,
            std::vector< std::vector< float > > & vec,
            char delimiter = ',')
```

Load a 2D vector of floats from a file.

Each non-empty line in the file is treated as a row in the 2D vector.

Parameters

out	*vec*	The 2D vector to be populated. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.5 load_vector_from_file() [3/3]

```
void load_vector_from_file (
            const std::string & filename,
            std::vector< std::vector< std::vector< float > > > & vec,
            char delimiter = ',')
```

Load a 3D vector from a file.

Reconstructs the 3D structure by using empty lines as separators between different 2D vectors.

Parameters

out	*vec*	The 3D vector to be populated. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.6 parseCSV()

```
std::vector< std::vector< std::string > > parseCSV (
            const std::string & filename)
```

Loading CSV file and parsing data into a string matrix.

It reads all lines - including the header.

Parameters

filename	The name of the CSV file to parse (".csv" must be included).

Returns

2D vector (rows and columns) of strings.

4.5.2.7 save_vector_to_file() [1/3]

```
void save_vector_to_file (
            const std::string & filename,
            const std::vector< float > & vec,
            char delimiter = ',')
```

Save a 1D vector of floats to a file.

Parameters

filename	Name of the file to create or overwrite.
vec	The 1D vector containing float data.
delimiter	The character used to separate values (default is comma).

4.5.2.8 save_vector_to_file() [2/3]

```
void save_vector_to_file (
            const std::string & filename,
            const std::vector< std::vector< float > > & vec,
            char delimiter = ',')
```

Save a 2D vector of floats to a file.

Each inner vector is written as a single line, with elements separated by the delimiter.

Parameters

vec	The 2D vector of floats. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.9 save_vector_to_file() [3/3]

```
void save_vector_to_file (
            const std::string & filename,
            const std::vector< std::vector< std::vector< float > > > & vec,
            char delimiter = ',')
```

Save a 3D vector of floats to a file.

2D vectors are separated by a blank line. Within each 2D vector, rows are separated by newlines and elements by the delimiter.

Parameters

vec	The 3D vector to save. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.6 file_data_operations.h

Go to the documentation of this file.

```
00001 #pragma once
00002 #ifndef FILE_DATA_OPERATIONS_H
00003 #define FILE_DATA_OPERATIONS_H
00007 #include <vector>
00008 #include <string>
00009 #include <fstream>
00010
00011 //Parsing functions
00017 std::vector<std::string> get_strings_from_line(const std::string& line);
00023 std::vector<float> get_floats_from_line(const std::string& line);
00024
00025 //Loading/saving to file functions
00032 std::vector<std::vector<std::string» parseCSV(const std::string& filename);
00033
00034 // Overload for 1D vectors
00041 void save_vector_to_file(const std::string& filename,
00042     const std::vector<float>& vec,
00043     char delimiter = ',');
00048 void save_vector_to_file(const std::string& filename, const std::vector<std::string>& vec, char
      delimiter = ',');
00049 // Overload for 2D vectors
00056 void save_vector_to_file(const std::string& filename,
00057     const std::vector<std::vector<float»& vec,
00058     char delimiter = ',');
00059 // Overload for 3D vectors
00067 void save_vector_to_file(const std::string& filename,const
      std::vector<std::vector<std::vector<float»>& vec,char delimiter = ',');
00068
00069 // Overloads for loading from file
00076 void load_vector_from_file(const std::string& filename, std::vector<std::string>& vec, char delimiter
      = ',');
00077
00078 // 2D vector overload
00085 void load_vector_from_file(const std::string& filename, std::vector<std::vector<float»& vec, char
      delimiter = ',');
00086 // 3D vector overload
00093 void load_vector_from_file(const std::string& filename, std::vector<std::vector<std::vector<float»>&
      vec, char delimiter = ',');
00094
00095
00096 #endif
```

4.7 NeuralNetworkProjectFoCP/math_functions.h File Reference

Declaration of various mathematical functions and their derivatives.

```
#include <vector>
#include <string>
#include <map>
#include <functional>
```

Classes

- struct ActivationFunction

  Wrapper for an activation function and its derivative. [.
- struct LossFunction

  Wrapper for a loss function and its derivative.

Functions

- std::vector< float > relu (const std::vector< float > &x)

  Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.
- std::vector< float > relu_derivative (const std::vector< float > &x)

  Derivative of the ReLU function: $f'(x) = 1$ *if* $x > 0$ *else* 0.
- std::vector< float > sigmoid (const std::vector< float > &x)

  Applies the sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.
- std::vector< float > sigmoid_derivative (const std::vector< float > &x)

  Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.
- std::vector< float > mtanh (const std::vector< float > &x)

  Applies the hyperbolic tangent (Tanh) activation: $f(x) = \tanh(x)$.
- std::vector< float > mtanh_derivative (const std::vector< float > &x)

  Applies the derivative of the Tanh activation on vector - 1 - tanh^2(x).
- std::vector< float > mean_squared_error (const std::vector< float > &predicted, const std::vector< float > &actual)

  Computes the Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.
- std::vector< float > mean_squared_error_derivative (const std::vector< float > &predicted, const std::vector< float > &actual)

  Computes the derivative of the MSE loss with respect to the prediction.
- std::vector< float > cross_entropy (const std::vector< float > &predicted, const std::vector< float > &actual)

  Binary Cross Entropy Loss: L = -[y∗log(p) + (1-y)∗log(1-p)].
- std::vector< float > cross_entropy_derivative (const std::vector< float > &predicted, const std::vector< float > &actual)

  Derivative of Binary Cross Entropy Loss with respect to predictions.

Variables

- const std::map< std::string, ActivationFunction > **activation_map**

  Maps string names (e.g., "relu") to ActivationFunction objects.
- const std::map< std::string, LossFunction > **loss_function_map**

  Maps string names (e.g., "MSE") to LossFunction objects.

4.7.1 Detailed Description

Declaration of various mathematical functions and their derivatives.

4.7.2 Function Documentation

4.7.2.1 cross_entropy()

```
std::vector< float > cross_entropy (
            const std::vector< float > & predicted,
            const std::vector< float > & actual)
```

Binary Cross Entropy Loss: L = -[y∗log(p) + (1-y)∗log(1-p)].

Parameters

predicted	The vector of predicted probabilities (must be in range 0-1).
actual	The true target labels (0 or 1).

Returns

A vector of cross-entropy losses.

4.7.2.2 cross_entropy_derivative()

```
std::vector< float > cross_entropy_derivative (
            const std::vector< float > & predicted,
            const std::vector< float > & actual)
```

Derivative of Binary Cross Entropy Loss with respect to predictions.

Parameters

predicted	The vector of predicted probabilities (must be in range 0-1).
actual	The true target labels (0 or 1).

Returns

Gradient vector.

4.7.2.3 mean_squared_error()

```
std::vector< float > mean_squared_error (
            const std::vector< float > & predicted,
            const std::vector< float > & actual)
```

Computes the Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.

Parameters

predicted	The output predicted by the network.
actual	The true target values.

Returns

A vector of squared differences.

4.7.2.4 mean_squared_error_derivative()

```
std::vector< float > mean_squared_error_derivative (
            const std::vector< float > & predicted,
            const std::vector< float > & actual)
```

Computes the derivative of the MSE loss with respect to the prediction.

$\frac{\partial L}{\partial pred} = pred - actual$

Parameters

predicted	The predicted output vector.
actual	The actual target vector.

Returns

Gradient vector.

4.7.2.5 mtanh()

```
std::vector< float > mtanh (
            const std::vector< float > & x)
```

Applies the hyperbolic tangent (Tanh) activation: $f(x) = \tanh(x)$.

Parameters

x	Input vector.

Returns

Vector with tanh values in range (-1, 1).

4.7.2.6 mtanh_derivative()

```
std::vector< float > mtanh_derivative (
            const std::vector< float > & x)
```

Applies the derivative of the Tanh activation on vector - 1 - tanh^2(x).

Parameters

x	Input vector.

Returns

Gradient vector.

4.7.2.7 relu()

```
std::vector< float > relu (
            const std::vector< float > & x)
```

Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.

Parameters

x	Input vector.

Returns

Vector with ReLU applied.

4.7.2.8 relu_derivative()

```
std::vector< float > relu_derivative (
            const std::vector< float > & x)
```

Derivative of the ReLU function: $f'(x) = 1$ if $x > 0$ else 0.

Parameters

x	Input vector.

Returns

Gradient vector.

4.7.2.9 sigmoid()

```
std::vector< float > sigmoid (
            const std::vector< float > & x)
```

Applies the sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.

Parameters

x	Input vector.

Returns

Vector with sigmoid values in range (0, 1).

4.7.2.10 sigmoid_derivative()

```
std::vector< float > sigmoid_derivative (
            const std::vector< float > & x)
```

Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.

Parameters

x	Input vector.

Returns

Gradient vector.

4.8 math_functions.h

Go to the documentation of this file.

```
00001 #pragma once
00005
00006 #include <vector>
00007 #include <string>
00008 #include <map>
00009 #include <functional>
00010
00016 std::vector<float> relu(const std::vector<float>& x);
00022 std::vector<float> relu_derivative(const std::vector<float>& x);
00023
00029 std::vector<float> sigmoid(const std::vector<float>& x);
00035 std::vector<float> sigmoid_derivative(const std::vector<float>& x);
00036
00042 std::vector<float> mtanh(const std::vector<float>& x);
00048 std::vector<float> mtanh_derivative(const std::vector<float>& x);
00049
00050
00057 std::vector<float> mean_squared_error(const std::vector<float>& predicted, const std::vector<float>&
      actual);
00065 std::vector<float> mean_squared_error_derivative(const std::vector<float>& predicted, const
      std::vector<float>& actual);
00066
00073 std::vector<float> cross_entropy(const std::vector<float>& predicted, const std::vector<float>&
      actual);
00080 std::vector<float> cross_entropy_derivative(const std::vector<float>& predicted, const
      std::vector<float>& actual);
00085 struct ActivationFunction {
00086     std::function<std::vector<float>(const std::vector<float>&)> func;
00087     std::function<std::vector<float>(const std::vector<float>&)> derivative;
00088 };
00093 struct LossFunction {
00094     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)> func;
00095     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)>
      derivative;
00096 };
00098 extern const std::map<std::string, ActivationFunction> activation_map;
00100 extern const std::map<std::string, LossFunction> loss_function_map;
```

4.9 NeuralNetworkProjectFoCP/neural_network.h File Reference

Core Neural Network class definition.

```
#include <vector>
#include <string>
#include <random>
```

Classes

- class NeuralNetwork

  Implementation of a neural network with multiple functionalities.

4.9.1 Detailed Description

Core Neural Network class definition.

4.10 neural_network.h

Go to the documentation of this file.

```
00001 #ifndef NEURAL_NETWORK_H
00002 #define NEURAL_NETWORK_H
00006 #include <vector>
00007 #include <string>
00008 #include <random>
00009
00014 class NeuralNetwork {
00015 public:
00016     NeuralNetwork();
00028     void initialize_weights_and_biases(int input_size, int hidden_layers_number, int
      neurons_per_hidden_layer, int output_size, std::string initialization_method = "Xavier",
00029         std::vector<std::string> activation_functions = {},
00030         int seed = std::random_device()());
00034     void visualize_model() const;
00041     std::vector<std::vector<float» feedforward(const std::vector<std::vector<float»& input) const;
00052     void train(const std::vector<std::vector<float»& inputs,
00053                const std::vector<std::vector<float»& targets,
00054                const std::vector<std::vector<float»& test_inputs,
00055                const std::vector<std::vector<float»& test_targets,
00056                int epochs,
00057                float learning_rate,
00058                const std::string& loss_function_name);
00066     float test_model(const std::vector<std::vector<float»& test_inputs,
00067                 const std::vector<std::vector<float»& test_targets,
00068                 const std::string& loss_function_name) const;
00073     void save_model_to_file(const std::string& filename) const;
00079     void load_model_from_file(const std::string& filename);
00080
00081     // Getters (mainly for testing)
00082     const std::vector<std::vector<std::vector<float»>& get_weights() const;
00083     const std::vector<std::vector<float»& get_biases() const;
00084     const std::vector<std::string>& get_activations() const;
00085
00086 private:
00087     std::vector<std::vector<std::vector<float»> weights;
00088     std::vector<std::vector<float» biases;
00089     std::vector<std::string> activations;
00090
00091     bool is_model_valid = false;
00092     void validate_model_structure();
00093
00094 };
00095 #endif
```

4.11 resource.h

```
00001 //{{NO_DEPENDENCIES}}
00002 // used by: NeuralNetworkProjectFoCP.rc
00003 //
00004 #define IDR_MENU1                       101
00005 #define ID_HEJ_HWEH                     40001
00006
00007 // Next default values for new objects
00008 //
00009 #ifdef APSTUDIO_INVOKED
00010 #ifndef APSTUDIO_READONLY_SYMBOLS
00011 #define _APS_NEXT_RESOURCE_VALUE        102
00012 #define _APS_NEXT_COMMAND_VALUE         40002
00013 #define _APS_NEXT_CONTROL_VALUE         1001
00014 #define _APS_NEXT_SYMED_VALUE           101
00015 #endif
00016 #endif
```

4.12 resource.h

```
00001 //{{NO_DEPENDENCIES}}
00002 // Microsoft Visual C++ generated include file.
00003 // Used by UnitTest1.rc
00004
00005 // Następne wartości domyślne dla nowych obiektów
00006 //
00007 #ifdef APSTUDIO_INVOKED
00008 #ifndef APSTUDIO_READONLY_SYMBOLS
00009 #define _APS_NEXT_RESOURCE_VALUE        101
00010 #define _APS_NEXT_COMMAND_VALUE         40001
00011 #define _APS_NEXT_CONTROL_VALUE         1001
00012 #define _APS_NEXT_SYMED_VALUE           101
00013 #endif
00014 #endif
```

4.13 pch.h

```
00001 // pch.h: wstępnie skompilowany plik nagłówka.
00002 // Wymienione poniżej pliki są kompilowane tylko raz, co poprawia wydajność kompilacji dla przyszłych
      kompilacji.
00003 // Ma to także wpływ na wydajność funkcji IntelliSense, w tym uzupełnianie kodu i wiele funkcji
      przeglądania kodu.
00004 // Jednak WSZYSTKIE wymienione tutaj pliki będą ponownie kompilowane, jeśli którykolwiek z nich
      zostanie zaktualizowany między kompilacjami.
00005 // Nie dodawaj tutaj plików, które będziesz często aktualizować (obniża to korzystny wpływ na
      wydajność).
00006
00007 #ifndef PCH_H
00008 #define PCH_H
00009
00010 // w tym miejscu dodaj nagłówki, które mają być wstępnie kompilowane
00011
00012 #endif //PCH_H
```